

Neo4J - Cypher Query Language

Student Table

sid	name	appno
1	Ananya	1001
2	Rohan	1002
3	Priya	1003

CSE Table

sid	cid	cname
1	201	Data Structures
2	202	Algorithms
3	203	Operating Systems

ECE Table

sid	cid	cname
1	301	Microprocessors
2	302	Digital Electronics
3	303	Communication Systems

USE Clause

:use university

MATCH (n) DETACH DELETE n

CREATE Clause

CREATE (gopi)

CREATE(gopi:person)

CREATE (s1:Student {sid: 1, name: 'Ananya', appno: 1001});

CREATE (s2:Student {sid: 2, name: 'Rohan', appno: 1002});

CREATE (s3:Student {sid: 3, name: 'Priya', appno: 1003});

CREATE (c1:CSE {sid: 1, cid: 201, cname: 'Data Structures'});

CREATE (c2:CSE {sid: 2, cid: 202, cname: 'Algorithms'});

CREATE (c3:CSE {sid: 3, cid: 203, cname: 'Operating Systems'});

CREATE (e1:ECE {sid: 1, cid: 301, cname: 'Microprocessors'});

CREATE (e2:ECE {sid: 2, cid: 302, cname: 'Digital Electronics'});

CREATE (e3:ECE {sid: 3, cid: 303, cname: 'Communication Systems'});

//CREATE Clause – Relationship

MATCH (s:Student {sid: 1}), (c:CSE {sid: 1})

CREATE (s)-[r:ENROLLED_IN]->(c);

MATCH (s:Student {sid: 2}), (c:CSE {sid: 2})

CREATE (s)-[:ENROLLED_IN]->(c);

```
MATCH (s:Student {sid: 3}), (c:CSE {sid: 3})
CREATE (s)-[:ENROLLED_IN]->(c);
```

```
MATCH (s:Student {sid: 1}), (e:ECE {sid: 1})
CREATE (s)-[:ENROLLED_IN]->(e);
```

```
MATCH (s:Student {sid: 2}), (e:ECE {sid: 2})
CREATE (s)-[:ENROLLED_IN]->(e);
```

```
MATCH (s:Student {sid: 3}), (e:ECE {sid: 3})
CREATE (s)-[:ENROLLED_IN]->(e);
```

MERGE Clause

Creating and Returning student Nodes (If already exists, it will update)

```
MERGE (s1:Student {sid: 1})
ON CREATE SET s1.name = 'Ananya', s1.appno = 2001
RETURN s1;
```

```
MERGE (s2:Student {sid: 2})
ON CREATE SET s2.name = 'Rohan', s2.appno = 1002
RETURN s2;
```

```
MERGE (s3:Student {sid: 3})
ON CREATE SET s3.name = 'Priya', s3.appno = 1003
RETURN s3;
```

CURD Operations

CREATE

Create: Adding Nodes and Relationships

```
CREATE (s:Student {sid: 1, name: 'Ananya', appno: 1001})
RETURN s;
```

```
MATCH (s:Student {sid: 1}), (c:CSE {cid: 201})
CREATE (s)-[:ENROLLED_IN]->(c)
RETURN s, c;
```

READ

Read: Querying Data from Nodes and Relationships

```
MATCH (n) RETURN n;
```

Selection

```
MATCH (s:Student {sid: 1}) RETURN s;
```

```
MATCH (s:Student)-[r:ENROLLED_IN]->(c:CSE)
RETURN s, r, c;
```

```
MATCH (s:Student)-[r:ENROLLED_IN]->(c:CSE{cname:'Data Structures'})
RETURN s.name;
```

UPDATE

SET Clause

Update: Modifying Existing Nodes and Relationships

Update Properties of a Node

```
MATCH (s:Student {sid: 1})
SET s.name = 'Ananya Sharma', s.appno = 1005
RETURN s;
```

Add a New Property to an Existing Node

```
MATCH (s:Student {sid: 1})
SET s.grade = 'A'
RETURN s;
```

Update Relationship Properties

```
MATCH (s:Student)-[r:ENROLLED_IN]->(c:CSE {cid: 203})
SET r.grade = 'A'
RETURN s,r,c;
```

```
MATCH (s:Student)-[r:ENROLLED_IN]->(c:CSE {cid: 203})
SET s.grade = 'A'
RETURN s,r,c;
```

DELETE

REMOVE Clause

//Removing a Property from a Node

```
MATCH (s:Student {name: 'Ananya Sharma'})  
REMOVE s.appno  
RETURN s;
```

// Removing a Label from a Node

```
MATCH (c:CSE {cid: 201})  
REMOVE c:CSE  
RETURN c;
```

// Removing Multiple Properties from a Node

```
MATCH (s:Student {sid: 2})  
REMOVE s.sid, s.name  
RETURN s;
```

// Removing a Label from Multiple Nodes

```
MATCH (e:ECE)  
REMOVE e:ECE  
RETURN e;
```

// Removing a Property from a Relationship

```
MATCH (:Student)-[r:ENROLLED_IN]->(:CSE)  
REMOVE r.enrollmentDate  
RETURN r;
```

// Removing a Label Conditionally

```
MATCH (s:Student)  
WHERE NOT EXISTS(s.appno)  
REMOVE s:Student  
RETURN s;
```

DELETE Clause

Delete: Removing Nodes and Relationships

Delete a Specific Node (Without Relationships) (You must first remove relationships or use DETACH DELETE.)

```
MATCH (s:Student {sid: 1})
```

```
DETACH DELETE s;
```

Delete a Specific Relationship

```
MATCH (s:Student)-[r:ENROLLED_IN]->(c:CSE {cid: 201})
```

```
DELETE r;
```

Delete All Nodes and Relationships

```
MATCH (n)
```

```
DETACH DELETE n;
```

RETURN Clause

// Projecting the `name` Property of All Students

```
MATCH (s:Student)
```

```
RETURN s.name AS studentName;
```

// Projecting Multiple Properties from All CSE Courses

```
MATCH (c:CSE)
```

```
RETURN c.cid AS courseID, c.cname AS courseName;
```

// Projecting Properties from Multiple Node Types

```
MATCH (s:Student), (c:CSE)
```

```
RETURN s.name AS studentName, s.appno AS applicationNumber, c.cid AS courseID,  
c.cname AS courseName;
```

// Projecting All Properties of a Node

```
MATCH (s:Student)
```

```
RETURN s;
```

// Projecting a Calculated Property

```
MATCH (s:Student)
```

```
RETURN s.sid AS studentID, s.appno + 1000 AS adjustedAppNo;
```

WHERE Clause

```
// Selecting Students Based on `appno`  
MATCH (s:Student)  
WHERE s.appno > 1001  
RETURN s;
```

```
// Selecting CSE Courses Based on `cid`  
MATCH (c:CSE)  
WHERE c.cid = 202  
RETURN c;
```

```
// Selecting Students by `name`  
MATCH (s:Student)  
WHERE s.name = 'Ananya'  
RETURN s;
```

```
// Selecting ECE Courses Based on `cname`  
MATCH (e:ECE)  
WHERE e.cname CONTAINS 'Digital'  
RETURN e;
```

```
// Selecting Courses Based on Multiple Attributes  
MATCH (c:CSE)  
WHERE c.cid = 201 AND c.cname STARTS WITH 'Data'  
RETURN c;
```

// Selecting Nodes Based on Multiple Conditions

MATCH (s:Student)

WHERE s.sid = 3 AND s.appno = 1003

RETURN s;

// Selecting Nodes Using Range Condition

MATCH (s:Student)

WHERE s.appno >= 1001 AND s.appno <= 1003

RETURN s;

// Selecting Students Enrolled in Both CSE and ECE Courses

MATCH (s:Student)-[:ENROLLED_IN]->(c:CSE), (s)-[:ENROLLED_IN]->(e:ECE)

WHERE c.cname = 'Data Structures' AND e.cname = 'Microprocessors'

RETURN s;

// Using OR to Select Students in Either of Two Specific Courses

MATCH (s:Student)-[:ENROLLED_IN]->(course)

WHERE (course:CSE AND course.cname = 'Data Structures') OR (course:ECE AND course.cname = 'Digital Electronics')

RETURN s;

// Selecting Students Not Enrolled in a Particular Course

MATCH (s:Student)-[:ENROLLED_IN]->(c:CSE)

WHERE NOT c.cname = 'Operating Systems'

RETURN s;

// Combining Multiple Conditions with AND and OR

MATCH (s:Student)-[:ENROLLED_IN]->(course)

WHERE s.appno > 1001 AND ((course:CSE AND course.cname = 'Algorithms') OR (course:ECE AND course.cname = 'Communication Systems'))

RETURN s;

[Creating a Complete Path](#)

Creating Paths:

// Match existing student nodes in the graph

MATCH (Ananya:Student {sid: 1}),

(Rohan:Student {sid: 2}),

(Priya:Student {sid: 3})

```
// Create a path connecting them with FRIEND_OF relationships
CREATE path = (Ananya)-[:FRIEND_OF]->(Rohan)-[:FRIEND_OF]->(Priya)
RETURN path;
```

```
//Creating a path with properties
CREATE path = (Ananya)-[:FRIEND_OF {since: 2021}]->(Rohan)-[:FRIEND_OF {since:
2022}]->(Priya)
RETURN path;
```

```
//Update to all the nodes in a path
MATCH path = (Ananya)-[*]->(Priya)
WHERE Ananya.name = "Ananya" AND Priya.name = "Priya"
FOREACH (n IN nodes(path) | SET n.college = "VIT")
RETURN path;
```

```
// Assigning the same marks value to all students
MATCH (s:Student)
SET s.marks = 85
RETURN s;
```

```
// Assigning different marks to each student based on `sid`
MATCH (s:Student)
SET s.marks = CASE s.sid
  WHEN 1 THEN 90
  WHEN 2 THEN 78
  WHEN 3 THEN 88
  ELSE 75 // Default marks if `sid` doesn't match any case
END
RETURN s;
```

```
// Using FOREACH to set marks on all students
MATCH (s:Student)
WITH collect(s) AS students
FOREACH (n IN students |
  SET n.marks = 85+n.sid
)
RETURN students;
```

Order By Clause

// Ordering Students by Marks in Ascending Order

MATCH (s:Student)

RETURN s.name, s.marks

ORDER BY s.marks;

// Ordering Students by Marks in Descending Order

MATCH (s:Student)

RETURN s.name, s.marks

ORDER BY s.marks DESC;

// Ordering by Multiple Properties (Marks and Year of Passing)

MATCH (s:Student)

RETURN s.name, s.marks, s.YOP

ORDER BY s.marks DESC, s.YOP ASC;

Limit Clause

// Limiting Results After Ordering

MATCH (s:Student)

RETURN s.name, s.marks

ORDER BY s.marks DESC

LIMIT 3;

// Ordering with Null Values Last

MATCH (s:Student)

```
RETURN s.name, s.marks
ORDER BY s.marks DESC NULLS LAST;
```

```
// Combining ORDER BY with Aggregation
MATCH (s:Student)
WITH s.YOP AS year, avg(s.marks) AS average_marks
RETURN year, average_marks
ORDER BY average_marks DESC;
```

Skip Clause

```
// Skipping the First Few Results
MATCH (s:Student)
RETURN s.name, s.marks
ORDER BY s.marks DESC SKIP 2;
```

```
// Using SKIP with LIMIT for Pagination
// Page 1 (First 3 students)
MATCH (s:Student)
RETURN s.name, s.marks
ORDER BY s.marks DESC LIMIT 3;
```

```
// Page 2 (Next 3 students)
MATCH (s:Student)
RETURN s.name, s.marks
ORDER BY s.marks DESC
SKIP 3 LIMIT 3;
```

```
// Skipping Results Based on Conditions
// Skip the first 2 students with marks above 80
MATCH (s:Student)
WHERE s.marks > 80
RETURN s.name, s.marks
ORDER BY s.marks DESC
SKIP 2;
```

```
// Using SKIP to Exclude the First Few Aggregated Results
```

```
// Skip the highest average marks by year
MATCH (s:Student)
WITH s.YOP AS year, avg(s.marks) AS average_marks
RETURN year, average_marks
ORDER BY average_marks DESC
SKIP 1;
```

Aggregate Functions

// 1. Calculating the Average Marks

```
MATCH (s:Student)
RETURN avg(s.marks) AS average_marks;
```

// 2. Finding the Total (Sum) of Marks

```
MATCH (s:Student)
RETURN sum(s.marks) AS total_marks;
```

// 3. Finding the Highest Marks (Max)

```
MATCH (s:Student)
RETURN max(s.marks) AS highest_marks;
```

// 4. Finding the Lowest Marks (Min)

```
MATCH (s:Student)
RETURN min(s.marks) AS lowest_marks;
```

// 5. Counting the Number of Students with Marks

```
MATCH (s:Student)
RETURN count(s.marks) AS number_of_students_with_marks;
```

// 6. Calculating the Standard Deviation of Marks

```
MATCH (s:Student)
RETURN stdev(s.marks) AS standard_deviation_marks;
```

// 7. Collecting Marks into a List

```
MATCH (s:Student)
```

```
RETURN collect(s.marks) AS marks_list;
```

// 8. Aggregating Multiple Statistics Together

```
MATCH (s:Student)
```

```
RETURN avg(s.marks) AS average_marks,  
       sum(s.marks) AS total_marks,  
       max(s.marks) AS highest_marks,  
       min(s.marks) AS lowest_marks,  
       stdev(s.marks) AS standard_deviation_marks,  
       count(s.marks) AS number_of_students_with_marks;
```

Count Function

The count() function is used to count the number of rows.

Syntax

```
MATCH (n { name: 'A' })-->(x)
```

```
RETURN n, count(*)
```

Example

To count all the node

```
MATCH ()
```

```
RETURN count(*) as count
```

To count all the relationship

```
MATCH ()-[r]->()
```

```
RETURN count(r) as count
```

To count all the nodes under the label

```
MATCH (:Person)
```

```
RETURN count(*) as count
```

To count the number of nodes have age under person label

```
MATCH (p:Person)
```

```
RETURN count(p.age)
```

To count based on a condition

```
MATCH (label) WHERE label.age >=50
```

```
RETURN count(*)
```

To count number of nodes have age 55

```
Match(n{age: 55})--(x)
RETURN n, count(*)
```

To count the number of relationships a person who have age 55

```
Match(n{age: 55})-[r]-(x)
RETURN type (r), count(*)
```

With Clause

// Using WITH for Intermediate Filtering

```
MATCH (s:Student)
WITH s
WHERE s.marks > 80
RETURN s.name, s.marks;
```

// Aggregating with WITH and Passing Results

```
MATCH (s:Student)
WITH s.appno AS year, avg(s.marks) AS average_marks
RETURN year, average_marks
ORDER BY average_marks DESC;
```

// Using WITH for Multiple Variables

```
MATCH (s:Student), (c:CSE)
WHERE s.sid = 1 AND c.cid = 201
WITH s, c
RETURN s.name AS student_name, c.cname AS course_name;
```

// Limiting Results with WITH and Chaining Queries

```
MATCH (s:Student)
WITH s
ORDER BY s.marks DESC
LIMIT 5
RETURN s.name, s.marks;
```

UNWIND Clause

```
// Expanding a List into Rows
UNWIND [1, 2, 3, 4, 5] AS number
RETURN number;
```

```
// Creating Nodes from a List
UNWIND ["Ananya", "Rohan", "Priya"] AS name
CREATE (s:Student {name: name})
RETURN s;
```

```
// Using UNWIND with Existing Data
MATCH (s:Student)
WITH s, [85, 90, 78] AS marksList
UNWIND marksList AS marks
RETURN s.name, marks;
```

```
// Combining WITH and UNWIND for Data Transformation
MATCH (s:Student)
WITH s, split(s.hobbies, ",") AS hobbiesList
UNWIND hobbiesList AS hobby
RETURN s.name, hobby;
```

```
// Aggregating Back After UNWIND
MATCH (s:Student)
WITH collect(s.name) AS studentNames
UNWIND studentNames AS name
```

```
RETURN name;
```

```
// Combined WITH and UNWIND Example
```

```
// Assign multiple marks to each student, unwind, and calculate the average
```

```
MATCH (s:Student)
```

```
WITH s, [85, 90, 78] AS marksList
```

```
UNWIND marksList AS mark
```

```
WITH s, avg(mark) AS average_marks
```

```
RETURN s.name, average_marks;
```

Predicate functions

Predicates are boolean functions that return true or false for a given set of non-null input. They are most commonly used to filter out subgraphs in the WHERE part of a query.

Create these nodes:

```
MERGE (Ajith:student:cse {name: "Ajith", roll:105, lang:['telugu','english','tamil']})
```

```
MERGE (Vijay:student:cse {name: "Vijay", roll:125, lang:['hindi','english','tamil']})
```

```
MERGE (Surya:student:cse {name: "Surya", roll:135, lang:['telugu','hindi']})
```

```
RETURN Ajith,Vijay,Surya
```

Functions:

```
all()
```

```
any()
```

```
exists()
```

```
none()
```

```
single()
```

```
all() :
```

```
MATCH (n:student:cse)
```

```
WHERE n.roll IS NOT NULL AND all(x IN [n.roll] WHERE x > 120)
```

```
RETURN n.name AS name, n.roll AS roll;
```

```
any() :
```

```
MATCH (n:student:cse)
```

```
WHERE n.lang IS NOT NULL AND any(x IN n.lang WHERE x = 'english')
```

```
RETURN n.name AS name, n.lang AS lang;
```

```
exists() :  
MATCH (n:student:cse)  
WHERE n.lang IS NOT NULL  
RETURN n.name AS name;
```

```
none() :  
MATCH (n:student:cse)  
WHERE n.lang IS NOT NULL AND none(x IN n.lang WHERE x = 'telugu')  
RETURN n.name AS name, n.lang AS lang;
```

```
single() :  
MATCH (n:student:cse)  
WHERE n.lang IS NOT NULL AND single(x IN n.lang WHERE x = 'english')  
RETURN n.name AS name, n.lang AS lang;
```

[Neo4j - Index](#)

Neo4j SQL supports Indexes on node or relationship properties to improve the performance of the application. We can create indexes on properties for all nodes, which have the same label name.

We can use these indexed columns on MATCH or WHERE or IN operator to improve the execution of CQL command.

Creating an Index

Syntax: CREATE INDEX ON:label (node)

Example:

CREATE

```
(s1:Student {id: 1, name: 'Alice', age: 20}),  
(s2:Student {id: 2, name: 'Bob', age: 22}),  
(s3:Student {id: 3, name: 'Carol', age: 21}),  
(s4:Student {id: 4, name: 'David', age: 23}),  
(s5:Student {id: 5, name: 'Eve', age: 20});
```

```
CREATE INDEX student_id_index FOR (s:Student) ON (s.id);
```

```
SHOW INDEXES;
```

Deleting an Index

Neo4j CQL provides a "DROP INDEX" command to drop an existing index of a Node or Relationship property.

Syntax: DROP INDEX ON:label(node)

Example: DROP INDEX student_id_index;

UNIQUE Constraint

Neo4j CQL provides "CREATE CONSTRAINT" command to create unique constraints on node or relationship properties.

Syntax

MATCH (root {name: "Dhawan"})

CREATE UNIQUE (root)-[:LOVES]-(someone)

RETURN someone

Example

CREATE CONSTRAINT student_id_unique FOR (s:Student) REQUIRE s.id IS UNIQUE;

SHOW CONSTRAINTS;

Neo4j CQL provides "DROP CONSTRAINT" command to delete existing Unique constraint from a node or relationship property.

Example

DROP CONSTRAINT student_id_unique;

String Functions

UPPER - It is used to change all letters into upper case letters.

 MATCH (n) RETURN toUpper(n.name)

LOWER - It is used to change all letters into lower case letters.

 MATCH (n) RETURN toLower(n.name)

SUBSTRING - It is used to get substring of a given String.

 MATCH (n) RETURN substring() (n.name,0,2)

Replace - It is used to replace a substring with a given substring of a String

 RETURN replace("hello", "l", "w")